

Maximum Number of Non-Overlapping Substrings

LeetCode 1520 · Hard

Greedy

Interval Closure

Two Pointers

Strings

1. Problem

Given a lowercase string `s`, select the maximum possible number of **non-overlapping** substrings such that every substring is **self-contained**: if a substring includes some character `c`, it must include *every* occurrence of `c` in the whole string. If multiple maximum-count solutions exist, return the one with the smallest total length (guaranteed unique). Substrings may be returned in any order.

Constraint	Value
Length of <code>s</code>	$1 \leq s.length \leq 1000$
Alphabet	lowercase English letters only
Return	An array of the chosen substrings (any order)

Important observations from the statement

- The "self-contained" rule can **cascade**: including one character might force you to also include another character's occurrences, which might in turn force a third — the boundary of a valid substring isn't fixed until this chain stops growing.
- "Maximum count, tie-broken by minimum total length" is a strong hint that **smaller, more nested** substrings should always be preferred over larger ones when both are available.

2. What Are We Really Being Asked?

For every letter, imagine the smallest window that starts at its first occurrence and is forced to keep growing rightward until it also contains every occurrence of every other letter that shows up inside it. Once that growing stops, you have a "closed" interval for that letter. The real question becomes: among all such closed intervals, pick as many **non-overlapping** ones as possible (with ties broken toward smaller total length).

Given	String <code>s</code>
Changing	Which letters' closed intervals are considered "reachable" as we scan; which intervals have already been claimed
To optimize	Maximize the count of non-overlapping self-contained substrings; break ties by minimizing total length
What makes it hard	A closed interval isn't known in advance — computing it requires simulating the cascade; and some naive per-letter ranges turn out to not be valid standalone intervals at all (they're swallowed by another letter's larger interval)

3. Key Observations

Observation 1 — Closure must expand, never just check

Start with each letter's naive `[first, last]` occurrence range. While scanning inside that range, any new letter encountered brings its own range along — if that range reaches further right, the window must grow to match. **Why it helps:** this converts "is this substring self-contained?" from a yes/no check into a constructive sweep that builds the correct boundary directly, one pass at a time.

Observation 2 — A range that needs to extend *left* of its assumed start is invalid on its own

If, while expanding letter `c`'s range, we meet a letter whose *first* occurrence lies before `start[c]`, then no valid self-contained interval can begin at `start[c]` — it would have to reach backward, contradicting its own starting point. **Why it helps:** this is exactly how the algorithm detects and discards "fake" candidate intervals — they aren't lost, they're simply absorbed into the other, earlier-starting letter's interval instead.

Observation 3 — Valid closed intervals are always nested or disjoint, never partially overlapping

If two closed intervals shared only part of their range, whatever letter caused the overlap would force both to merge into one larger closed interval (by Observation 1) — so that situation can never persist. **Why it helps:** this structural guarantee is what allows a simple single right-to-left sweep to correctly pick a maximal, non-overlapping, minimum-length set — no general-purpose interval-scheduling sort or DP is needed.

4. Intuition

Solve it in two stages.

Stage 1 — build every letter's closed interval. For letter `c`, start at `start[c]` and sweep right. Every letter you pass over must have its entire range swallowed too — if that pushes the end further out, keep going until the sweep naturally catches up to the (possibly expanded) end. If you ever discover a letter whose first occurrence is further left than where you started, this interval can never be valid standalone — mark it invalid and move on (Observation 2).

Stage 2 — greedily select intervals, right to left. Once every closed interval is known, walk the string backward. Whenever the current position is exactly the start of a valid, still-unclaimed interval, and that interval doesn't reach into territory already claimed by something picked more recently, take it. Because closed intervals are always nested or disjoint (Observation 3), sweeping right to left naturally lets inner, smaller intervals get first pick over the outer intervals that contain them — exactly matching the "prefer smaller, more specific substrings" requirement from the problem.

One-line mental model: "First figure out how far each letter's influence really spreads (closure). Then, scanning from the end, greedily grab the most nested interval available at each point, letting bigger, redundant outer intervals lose out."

5. Approach — Interval Closure + Right-to-Left Greedy Selection

Only one approach was provided in the source code — no separate brute-force section is included since none was supplied. This is the approach actually used.

5.1 Idea

1. **Closure pass:** for each of the 26 letters, compute its naive `[start, end]` range, then sweep through that range, expanding `end` whenever a contained letter's own range reaches further, and marking the letter invalid if a contained letter's `start` lies before this letter's own start.
2. **Selection pass:** scan the string right to left; greedily take any valid interval whose start we land on, as long as it doesn't overlap the most recently taken interval.

5.2 Why it works

The closure pass guarantees every surviving interval is genuinely self-contained (Observation 1), and correctly discards intervals that can't stand alone (Observation 2). The nesting-or-disjoint structure (Observation 3) means a single right-to-left greedy pass — always preferring the interval you reach first while scanning backward — is enough to build an optimal non-overlapping set: taking an inner interval never costs you the option of also taking whatever remains outside it, and skipping an outer interval whose territory is already spoken for never loses anything, since it wasn't a valid additional choice anyway.

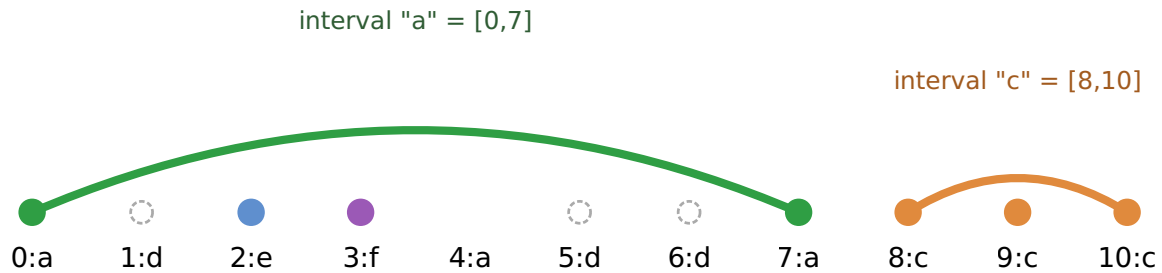
5.3 Pseudocode

```
// Stage 1: closure – find each letter's self-contained interval
for i in 0..n-1:
    start[s[i]] = min-or-first-seen(i)
    end[s[i]] = i                                // keeps updating to the latest occurrence

for c in 'a'..'z':
    if start[c] undefined: continue
    i = start[c]
    while i < end[c]:
        c2 = s[i]
        if start[c2] < start[c]:
            isValid[c] = false
            break
        end[c] = max(end[c], end[c2])
    i++

// Stage 2: greedy selection, right to left
lastTakenStart = n
for i = n-1 downto 0:
    c = s[i]
    if not isValid[c]: continue
    if i == start[c] and end[c] < lastTakenStart:
        take substring s[start[c] .. end[c]]
        lastTakenStart = start[c]
```

5.4 Visual — closure & nesting for `s = "adefaddaccc"`



■ closed interval "a" [0,7] ■ closed interval "c" [8,10] ■ "e" [2,2] (nested) ■ "f" [3,3] (nested) □ "d" — invalid, absorbed into "a"

Notice "e" and "f" sit strictly inside "a"'s interval (nested, not overlapping), and "c" sits completely outside it (disjoint) — exactly the structure guaranteed by Observation 3.

5.5 Time and space complexity

Stage	Time	Space	Notes
Closure (26 letters)	$O(26 \cdot n) = O(n)$	$O(26) = O(1)$	each letter's sweep only ever touches its own final range once
Greedy selection	$O(n)$	$O(n)$	single backward pass; space for the output substrings
Total	$O(n)$	$O(n)$	the constant factor of 26 is dropped as it's independent of input size

5.6 Implementation (as provided, unmodified)

```
vector<string> maxNumOfSubstrings(string s) {
    int n = s.size();
    vector<int> start(26, -1);
    vector<int> end(26, -1);
    vector<bool> isValid(26, true);

    // naive first/last occurrence per letter
    for (int i = 0; i < n; i++) {
        if (start[s[i] - 'a'] == -1) start[s[i] - 'a'] = i;
        end[s[i] - 'a'] = i;
    }

    // closure: expand each letter's range to be self-contained
    for (int c = 0; c < 26; c++) {
        if (start[c] == -1) continue;
        for (int i = start[c]; i < end[c]; i++) {
            if (start[s[i] - 'a'] < start[c]) {
                isValid[c] = false;
                break;
            }
        }
    }
}
```

```

        end[c] = max(end[c], end[s[i] - 'a']);
    }
}

// greedy, right to left
vector<string> ans;
int lastTakenStart = n;
for (int i = n - 1; i >= 0; i--) {
    int idx = s[i] - 'a';
    if (!isValid[idx]) continue;
    if (i == start[idx] && end[idx] < lastTakenStart) {
        ans.push_back(s.substr(i, end[idx] - i + 1));
        lastTakenStart = start[idx];
    }
}
return ans;
}

```

Design note: the closure loop's condition `i < end[c]` is re-evaluated fresh on every iteration in C++, so when `end[c]` grows mid-loop the sweep automatically continues further — no explicit "restart the scan" logic is needed, which is what keeps this a single linear pass per letter instead of a repeated fixed-point loop.

6. Detailed Dry Run

Test case: `s = "adefaddaccc"` (expected: `{"e", "f", "ccc"}`, any order)

6.1 Closure results

Letter	start	end (after closure)	isValid	Why
a	0	7	true	sweep absorbs d, e, f but nothing has a start < 0
d	1	6	false	sweep hits 'a' at index 4, and <code>start[a]=0 < start[d]=1</code> → invalid, absorbed into a's interval instead
e	2	2	true	single occurrence, trivially self-contained
f	3	3	true	single occurrence, trivially self-contained
c	8	10	true	all three occurrences are already adjacent; nothing else to absorb

6.2 Right-to-left selection trace

i	char	Action	lastTakenStart after
10, 9	c, c	not at <code>start[c]=8</code> yet — skip	11 (n)
8	c	<code>i==start[c]</code> and <code>end[c]=10 < 11</code> → take "ccc"	8
7	a	<code>i≠start[a]=0</code> — skip	8

6, 5	d, d	invalid — skip	8
4	a	$i \neq \text{start}[a]=0$ — skip	8
3	f	$i == \text{start}[f]=3$ and $\text{end}[f]=3 < 8 \rightarrow$ take "f"	3
2	e	$i == \text{start}[e]=2$ and $\text{end}[e]=2 < 3 \rightarrow$ take "e"	2
1	d	invalid — skip	2
0	a	$i == \text{start}[a]=0$, but $\text{end}[a]=7$ is not $< 2 \rightarrow$ skip (would overlap "e"/"f")	2

Result: ["ccc", "f", "e"] — three substrings, matching the expected count and content (order doesn't matter per the problem statement).

7. Test Cases

Case	Input	Output	Explanation
Normal case	"adefaddaccc"	{"e", "f", "ccc"}	traced in full above; "d" gets absorbed into "a"
Normal case	"abbaccd"	{"d", "bb", "cc"}	"a" absorbs "b" (nested), "c" and "d" stand alone
Smallest case	"a"	{"a"}	a single character is trivially self-contained
Everything entangled	"abab"	{"abab"}	"a"'s range and "b"'s range each force the other to expand to cover the whole string — closure collapses to one interval
All-same-letter case	"aaaa"	{"aaaa"}	only one letter exists, so the whole string is its single closed interval

8. Why the Solution Works

The closure pass is correct because it directly simulates the containment rule: it never marks a range "done" until every letter inside it has been checked, and any letter that would force the boundary backward is correctly flagged invalid rather than silently producing a broken interval. This leaves only genuinely self-contained candidate intervals.

The selection pass is correct because of the nesting-or-disjoint guarantee (Observation 3): since no two valid intervals partially overlap, there's never an ambiguous case where taking one interval "almost" fits with another. Scanning right to left and greedily taking any interval reached — as long as it doesn't intrude on space already claimed — necessarily favors inner intervals over the outer intervals that contain them, since inner intervals are encountered (and satisfy `i == start[idx]`) before the sweep reaches the outer interval's start. This directly produces both the maximum count (every nested interval that can be taken, is taken) and the minimum total length (smaller nested intervals are always chosen over the larger interval that contains them).

9. Common Mistakes

Mistake	Why it's wrong
Treating every letter's naive <code>[first, last]</code> range as a valid substring directly	Ignores the containment cascade — a range like "d"'s [1,6] can secretly need to include letters (like "a") that start even earlier, making it invalid on its own
Overwriting <code>end[c]</code> instead of taking a <code>max</code> during closure	Could shrink the boundary back down if a later-checked contained letter has a smaller end, silently under-covering required occurrences
Selecting intervals left to right instead of right to left	Left-to-right greedy would grab an outer interval (like "a"'s [0,7]) before discovering the smaller nested intervals inside it, violating the "minimum total length" tie-break
Forgetting to check <code>isValid</code> during selection	Would incorrectly treat an absorbed/invalid range (like "d") as a standalone answer, producing overlapping or incorrect substrings
Assuming substring order in the output matters	The problem explicitly allows any order; forcing a specific order (e.g. left-to-right) isn't wrong, but assuming the grader requires it can lead to unnecessary extra sorting logic

10. Pattern Recognition

How to recognize this type of problem again:

- Phrases like *"if a substring contains X, it must contain all occurrences of X"* → think **closure over first/last-occurrence ranges**, expanding until stable (the same core idea behind the simpler LeetCode 763, "Partition Labels").
- When a closure-style construction guarantees resulting intervals are always **nested or disjoint** (never partially overlapping) → a single greedy sweep suffices; no need for a full weighted interval-scheduling DP.
- "Maximum count, ties broken by minimum total length/size" → a strong signal that greedily preferring smaller/more nested valid options over larger ones is provably optimal.
- A scan that must possibly "look backward" to invalidate a candidate (as in Observation 2) → consider whether a right-to-left pass simplifies the selection logic more than a left-to-right one would.

11. Final Takeaway

CORE IDEA: Compute each letter's self-contained ("closed") interval by expanding its naive first/last-occurrence range until stable, discard ranges that would need to expand backward, then greedily take non-overlapping valid intervals scanning right to left.

KEY OBSERVATION: Closed intervals are always nested or disjoint, never partially overlapping — this is what makes a simple right-to-left greedy sweep sufficient and optimal.

COMPLEXITY: Time $O(n)$ ($O(26 \cdot n)$ with a constant alphabet size), Space $O(n)$.

PATTERN: First/last-occurrence closure (same family as "Partition Labels") + right-to-left greedy selection exploiting a nesting-or-disjoint interval structure.